

FINAL PROJECT REPORT

Implement Client Script & UI Policy (Incident)

Team ID	SWTID-2026-9644
Team Structure	Vairavakarumari A (Team Leader) Abinaya K (Member) Kayathri Devi B (Member) Team Size: 3
Project Domain	ServiceNow ITSM / Incident Management / Client-Side Form Controls
Submission Date	30 September 2026

1. INTRODUCTION

1.1 Project Overview

Incident management requires accurate and consistent information so that support teams can triage, route, prioritize, and resolve issues effectively. This project demonstrates a ServiceNow client-side solution using UI Policies and Client Scripts to improve the quality of Incident records at the point of entry.

1.2 Purpose

The primary purpose is to enforce defined Incident data-entry rules directly on the user interface. The implementation controls field visibility and mandatory status, auto-populates values where required, and validates records before submission so incomplete or invalid Incidents can be prevented.

2. IDEATION PHASE

2.1 Problem Statement

Incident records often require consistent and accurate data entry to ensure effective triage, routing, and resolution. Relying solely on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect data being submitted. This can affect reporting accuracy, SLA compliance, and overall service quality. Standardized UI Policies and Client Scripts are therefore needed to enforce conditional field behavior and save-time validation.

2.2 Empathy Map Canvas

- Says & Does:** Users want clear required fields and a simple Incident form; they create, update, and submit Incident records.
- Thinks & Feels:** Users may be concerned about missing required information and frustrated by validation errors or repeated corrections.
- Pains:** Incomplete records, inconsistent data, repetitive manual checking, and incorrect routing caused by missing information.
- Gains:** Guided data entry, consistent records, dynamic mandatory fields, auto-populated values, and validation before submission.

2.3 Brainstorming

During ideation, the team identified several client-side interventions: (1) conditional UI Policies for field visibility and mandatory status; (2) Client Scripts for auto-population and form validation; (3) save-time validation to prevent submission when required conditions are not satisfied; and (4) standardized behavior to improve Incident data consistency.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

1. Open Incident Form → 2. Enter Incident Details → 3. Dynamic Field Control → 4. Client Validation → 5. Submit → 6. Valid Incident Record

User opens the Incident form • Enters relevant information • UI Policies dynamically control fields and mandatory status • Client Scripts populate or validate values • Invalid submissions are stopped • Valid records are submitted for triage and routing.

3.2 Solution Requirements

Functional: Conditional mandatory fields, field visibility/control, client-side auto-population, save-time validation, and prevention of invalid submissions.

Non-Functional: Consistent user experience, immediate form feedback, maintainable client-side rules, and improved Incident data quality.

3.3 Data Flow Description

[Service Desk User] → (Enter Incident Data) → [ServiceNow Incident Form] → [UI Policy / Client Script] → (Validate & Control Fields) → [Valid Incident Record] → [Triage / Routing]

3.4 Technology Stack

Layer	Technologies & Tools Used
Core Platform	ServiceNow ITSM / Incident Management
Client-Side Controls	UI Policies, Client Scripts
Form Validation	onChange / onSubmit client-side validation logic
Incident Data	ServiceNow Incident table (incident)

4. PROJECT DESIGN

4.1 Problem Solution Fit

The solution directly addresses incomplete and inconsistent Incident data by moving important checks to the user interface. Users receive immediate field guidance and validation before a record is submitted.

4.2 Proposed Solution

Implement UI Policies to dynamically make relevant fields mandatory or visible based on Incident conditions, and Client Scripts to auto-populate values and validate required information before submission.

4.3 Solution Architecture

Three logical layers are used: Presentation Layer (Incident form), Business Rule Control Layer (UI Policies and Client Scripts), and Data Layer (ServiceNow Incident record).

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning (Sprint / Phase Breakdown)

Phase	Key Deliverables	Assigned To	Duration / Status
Phase 1	Requirements gathering, Incident field analysis & business rules	Vairavakarumari A (Lead)	Weeks 1–2
Phase 2	UI Policy design and conditional field behavior	Abinaya K	Weeks 3–4
Phase 3	Client Scripts, auto-population & validation	Kayathri Devi B	Weeks 5–6
Phase 4	Integration of controls, testing & UAT	Team	Weeks 7–8

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Functional Testing

Test ID	Scenario	Test Steps & Input	Expected Result	Result
FT-01	Conditional Mandatory Field	Set Incident condition that activates a required field	Field becomes mandatory	PASS
FT-02	Field Visibility / Control	Change controlling Incident value	Configured field behavior changes dynamically	PASS
FT-03	Auto-Population	Trigger configured Client Script condition	Target value is populated automatically	PASS
FT-04	Submit Validation	Leave required condition/data incomplete and submit	Submission is prevented with user feedback	PASS

6.2 Performance / Usability Testing

- UI changes should occur immediately when controlling field values change.
- Client-side validation should provide clear feedback before record submission.
- Configured policies and scripts should not create unnecessary form interaction.
- Valid Incident records should be submitted successfully without manual correction.

7. RESULTS

7.1 Expected Output

ServiceNow Incident form with dynamically controlled fields; required fields enforced according to conditions; configured values auto-populated where applicable; invalid submissions prevented; and valid Incidents created with consistent information.

8. ADVANTAGES & DISADVANTAGES

Advantages:

- Improves completeness and consistency of Incident records.
- Reduces manual checking and correction effort.
- Provides immediate guidance through dynamic field behavior.
- Supports better triage, routing, reporting, and SLA-related data quality.
- Prevents submission when configured validation conditions are not met.

Disadvantages / Constraints:

- Client-side controls mainly affect the user interface and should not be treated as a replacement for server-side controls where required.
- Incorrectly configured policies or scripts can affect the user experience.
- Rules need maintenance when Incident form fields or business requirements change.

9. CONCLUSION

The Implement Client Script & UI Policy (Incident) project demonstrates how ServiceNow client-side controls can improve Incident data integrity. By using UI Policies and Client Scripts, the solution dynamically controls field behavior, makes fields mandatory when required, auto-populates values, and validates records before submission. This supports the creation of complete and valid Incident records and improves consistency across the platform.

10. FUTURE SCOPE

- Extend validation with additional Incident business scenarios and user roles.
- Introduce reusable client-side validation patterns for other ITSM forms.
- Combine client-side controls with appropriate server-side validation for stronger data integrity.
- Add richer user guidance and contextual messages for complex Incident conditions.
- Measure data-quality improvements using Incident reporting and trend analysis.

11. APPENDIX

Configuration Items: ServiceNow UI Policies, Client Scripts, Incident form fields and validation conditions.

Project Team: Vairavakarumari A (Team Leader), Abinaya K, Kayathri Devi B.

Team ID: SWTID-2026-9644

Project: Implement Client Script & UI Policy (Incident)